Plates — Hybrid Session Data Model

Nov 4, 2026 · @Maxwell Jefferson

Overview

One day can hold any mix of lifting, track work and running, and every piece rolls up into one Day Load number that drives that day's fuel targets.

Design principles:

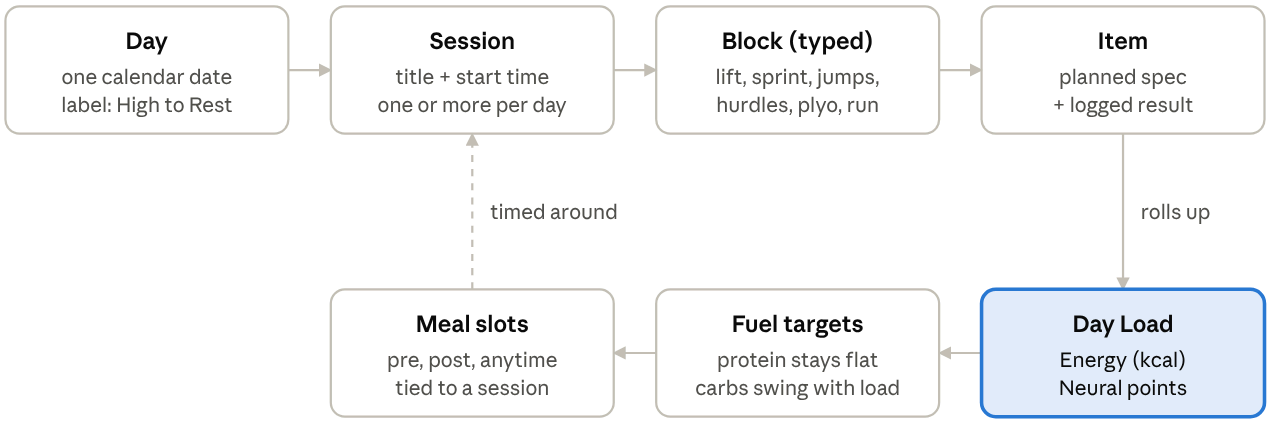
- **One timeline, many modalities.** A Day holds ordered Sessions; a Session holds Blocks; each Block has a type (lift, sprint, jumps, hurdles, plyo, run, conditioning). A hybrid day is just two or more blocks.
- **Plan and actual live side by side.** Every prescribed item has an optional logged result, so the Week view, adherence and the Plates Score all read from the same records.
- **Track is first-class, not a note field.** Distances, times, marks, hurdle specs and ground contacts are typed fields, so they can be charted and fed into load.
- **Import, don't rebuild.** Runs come from Apple Health or Strava. Plates doesn't record GPS.
- **Same model on phone and web.** The website edits the same records, so nothing in the model can be iOS-only.

Core hierarchy

Day → Session → Block → Item is the whole training structure; everything else hangs off it.

Every logged item rolls up into Day Load, which sets the fuel targets

Top row: training structure. Bottom row: what it drives.



data model · training structure and what it drives

A Week is just seven Days (no stored entity). A reusable Routine (today's "Pull day" card) is a saved Session template that gets copied onto a Day when assigned.

Block types and their fields

Every block shares the same envelope. Only the item rows inside it change by type.

Shared block envelope: id, sessionId, type, order, title, notes, plannedStart, rpeTarget, rpeActual, completedAt.

Shared item fields: id, blockId, order, setCount, restSec, planned (typed per block), actual (typed per block, optional), isSkipped, notes.

Block type	Item =	Planned fields	Logged fields	Load inputs
Lift	one exercise	exerciseId, sets × reps range, load (lb/kg or %1RM), tempo, RIR target, progression rule	per set: weight, reps, RIR, isWarmup	sets × reps × load (tonnage), hard sets

Block type	Item =	Planned fields	Logged fields	Load inputs
Sprint	one rep scheme	distance (m), reps, sets, intensity (% of PR or effort tier), start type (blocks, 3-pt, fly, standing), surface, footwear (spikes/flats), rest	per rep: time (s), timing method (hand/FAT/laser/camera), wind (m/s), felt quality 1–5	meters at ≥ 90% intensity, total meters
Hurdles	one drill or rep	hurdle count, height (in/cm), spacing (m), approach (strides to H1), reps, intensity	per rep: time to H1, touchdown times, total time, hits/knocks	meters at speed + hurdle count
Jumps	one jump series	event (LJ, TJ, HJ, PV), approach (full, short, # of strides), attempts, takeoff board/zone	per attempt: mark (m), foul y/n, approach speed if measured, wind	takeoffs × approach length
Plyo	one drill	drill (pogo, bound, depth jump…), contacts per set, sets, box height	contacts done, RSI or height if measured	ground contacts, weighted by intensity tier
Run	one run or interval set	type (easy, tempo, long, intervals, race), distance or duration, target pace or HR zone, interval reps × distance	distance, duration, avg pace, avg HR, elevation, source (manual, Health, Strava)	distance × intensity zone

Block type	Item =	Planned fields	Logged fields	Load inputs
Conditioning	one piece	format (EMOM, circuit, sled, bike), work/rest, rounds	rounds done, avg HR, notes	duration × RPE

Two shared libraries back these:

- **Exercise library:** lifts, drills and plyos, each with a category, default load units and a CNS intensity tier (low, moderate, high).
- **Event library:** 60, 100, 200, 400, the hurdle events, the jumps, plus road distances. PRs and % of PR intensities key off these.

Day Load

Each day gets two numbers, not one: **Energy** (how much fuel the work burns) and **Neural** (how hard it hits the CNS). A 6 × 30 m fly day is low energy but high neural; a 10-mile tempo run is the reverse. One blended score would hide exactly the difference that matters for sprinters.

Energy (kcal), per block, using the first source available:

1. Wearable active energy for the block's time window (Apple Watch, WHOOP via Health).
2. Imported run calories (Strava or Health).
3. Estimate: duration × MET for the block type × body weight.

```
\text{kcal}_{block} = \text{MET}_{type} \times \text{kg} \times
\frac{\text{minutes}}{60}
```

Neural load points, per block: each type turns its load inputs into points with tunable weights. Starting defaults, meant to be adjusted once there's real data:

Input	Points
Sprint meters at ≥ 90% intensity	1 per 10 m
Full-approach jump takeoffs	4 each (short approach: 2)
Hurdle reps at race spacing	3 each

Input	Points
Plyo ground contacts	0.1 / 0.25 / 0.5 per contact by tier
Lift hard sets (≤ 3 RIR) on high-tier lifts	2 per set
Lift hard sets on other lifts	0.5 per set
Run time in zone 4–5	1 per 5 min

Day label: Neural points sum to High, Moderate, Low or Rest by thresholds stored per user. This is the High/Low CNS day split, and it's what the Week view colors days by.

Planned load is computed from the plan; actual load recomputes from logged results, so a cut-short session lowers that day's targets.

Fuel link

Daily targets = the user's base (goal, body weight, weekly adjustment) + the day's Energy, split by macro rules that read the day label.

1. **Base targets** come from the goal (cut, maintain, lean gain) and update weekly from logged weight and intake. This is the existing "targets adjust each Monday" logic.
2. **Calories** = base + a share of that day's planned Energy. Rest days use base only.
3. **Protein** stays flat across the week (g per kg of body weight). Recovery doesn't take days off.
4. **Carbs** take most of the swing: high-Energy days get more, rest days less. Fat fills the rest.
5. **Timing.** Meals carry a role relative to a session (pre, intra, post, or anytime) instead of only a clock time. Moving a session from 7 AM to 5 PM moves its pre and post meals with it.

Two fixes from the current build fall out of this:

- When the planned meals don't reach the day's target (e.g. 2,070 planned vs 2,180), show the gap and suggest a meal or snack to close it.
- Meal slots get unique roles, so two "Breakfast 8:00 AM" items become, for example, Breakfast + Pre-lift.

Entities: DayTargets (date, kcal, protein, carbs, fat, source = computed or manual override), MealPlan, MealSlot (dayId, role, sessionRef, time), MealItem (foodId or recipeId, servings, isEaten).

PRs and performance records

A PR is a stored Performance row, not something recalculated on the fly, so meet results and training bests sit on the same chart.

Performance: id, userId, eventId or exerciseId, value, unit (s, m, lb, kg), date, context (meet, training, test), timing (hand, FAT, laser, camera), wind (m/s), isWindLegal (computed: wind ⩽ +2.0 m/s for sprints, hurdles and horizontal jumps), source logId, meetName, notes.

Rules:

- **Best marks are scoped.** "PR" means the best wind-legal FAT mark from a meet; training, hand-timed and wind-aided bests show as separate badges. Track athletes care about this distinction.
- **Lifts get estimated 1RMs.** Each logged set updates an e1RM, and a true 1RM test is stored as its own Performance row.
- **% of PR flows back into planning.** A sprint block planned at 95% of a 30 m fly PR shows the target time on the logging screen.
- **A PR is a moment.** A new PR triggers a shareable card (event, mark, wind, date). That's the screen track athletes will post.

Imports, wearables, sync and storage

Use a Supabase (Postgres) backend as the source of truth, with a local SwiftData cache on the phone, so the iPhone app and the website share one database and the app still works offline.

Source	What comes in	Lands as
Apple Health (HealthKit)	workouts, active energy, HR, body weight, sleep	Run blocks (auto-matched to a planned run by date and type), energy for any block's window, weight for targets

Source	What comes in	Lands as
Strava (OAuth)	runs with distance, pace, HR, splits	Run blocks; the website can read these too, which HealthKit can't
WHOOP (via Health or its API)	strain, recovery, sleep	readiness inputs; a later phase
Manual	everything else	as typed

Matching rule: an import within ±3 h of a planned Run block of the same type fills that block's actual fields. Otherwise it becomes a new unplanned block on that day, which still counts toward load.

Sync: every table gets id (UUID), userId, updatedAt and deletedAt (soft delete). The phone writes locally first, then pushes; conflicts resolve last-write-wins per row, which is fine for one person editing on two devices. Supabase Row Level Security keeps each user's rows private, and coach access later becomes an RLS policy instead of a rewrite.

This replaces the earlier CloudKit plan, which can't serve a website or Android cleanly.

Swift model sketch

The per-type fields go in one Codable enum, stored as a JSONB column in Postgres. Adding a new block type then means one new case, not a new table.

```
enum BlockType: String, Codable { case lift, sprint, hurdles, jumps, plyo,
run, conditioning }
enum Intensity: String, Codable { case low, moderate, high }
enum DayLabel: String, Codable { case high, moderate, low, rest }

struct Session: Identifiable, Codable {
    let id: UUID
    var dayDate: Date           // the calendar day
    var title: String           // "Speed + Lower"
    var plannedStart: Date?
    var order: Int
    var blocks: [Block]
}

struct Block: Identifiable, Codable {
    let id: UUID
```

```
    var type: BlockType
    var order: Int
    var title: String
    var rpeTarget: Double?
    var rpeActual: Double?
    var items: [BlockItem]
}

struct BlockItem: Identifiable, Codable {
    let id: UUID
    var order: Int
    var restSec: Int?
    var planned: ItemSpec        // what the coach/athlete prescribed
    var actual: ItemResult?      // nil until logged
}

enum ItemSpec: Codable {
    case lift(exerciseId: UUID, sets: Int, repsMin: Int, repsMax: Int, load:
Load, rirTarget: Int?)
    case sprint(distanceM: Double, reps: Int, sets: Int, pctOfPR: Double?,
start: StartType, footwear: Footwear)
    case hurdles(count: Int, heightIn: Double, spacingM: Double,
approachStrides: Int, reps: Int)
    case jumps(event: JumpEvent, approachStrides: Int, attempts: Int)
    case plyo(drillId: UUID, contactsPerSet: Int, sets: Int, tier: Intensity)
    case run(kind: RunKind, distanceM: Double?, durationSec: Int?,
targetPaceSecPerKm: Int?)
    case conditioning(format: String, workSec: Int, restSec: Int, rounds:
Int)
}

enum ItemResult: Codable {
    case lift(sets: [LiftSet])                // weight, reps, rir, isWarmup
    case sprint(reps: [TimedRep])             // seconds, timing, windMS
    case hurdles(reps: [HurdleRep])           // toH1, touchdowns, total,
hits
    case jumps(attempts: [JumpAttempt])       // markM, isFoul, windMS
    case plyo(contacts: Int, rsi: Double?)
    case run(distanceM: Double, durationSec: Int, avgHR: Int?, source:
DataSource)
    case conditioning(rounds: Int, avgHR: Int?)
}
```

```
struct DayLoad: Codable {
    var date: Date
    var energyKcal: Double
    var neuralPoints: Double
    var label: DayLabel
    var isPlanned: Bool            // planned vs actual
}
```

The load functions (`func load(for block: Block) -> (kcal: Double, neural: Double)`) live in a shared module, so the same rules can later be ported to the website as one TypeScript file.

v1: TestFlight to teammates by Dec 1

The goal is for teammates on winter break to log their coach's break program (sprints, jumps, hurdles, plyos, lifts) and see fuel targets that follow it. Submit the build by Mon Nov 23, since external TestFlight needs Apple's beta review and Thanksgiving is Nov 26.

Week 1 (Nov 4–10): foundation

- [ ] Supabase project, Sign in with Apple, Row Level Security
- [ ] Session → Block → Item model in SwiftData + sync
- [ ] Migrate current lift and meal screens onto the new model

Week 2 (Nov 11–17): track logging

- [ ] Sprint, Jumps, Hurdles and Plyo block types with fast logging screens
- [ ] Copy a session or a whole week (break programs repeat)
- [ ] Basic PR list per event (wind-legal flag included)

Week 3 (Nov 18–23): personalization + fuel

- [ ] Onboarding flow (below) and starter-week templates
- [ ] Day Load v1: Neural day label + estimated Energy
- [ ] Fuel targets from Day Load; Open Food Facts barcode + USDA search
- [ ] Fix: 132 vs 88 lb mismatch, planned-vs-target gap, duplicate meal slots
- [ ] Submit to TestFlight beta review (Nov 23)

Week 4 (Nov 24–30): buffer

- [ ] Bug fixes from your own use, a feedback button in the app, invite teammates

Not in v1: website, Coach tier, Strava, WHOOP, share cards, meal plans and grocery lists.

Onboarding

Six short screens, and the answers decide which blocks, Home cards and fuel detail each user sees.

1. **What do you train?** (pick any) Sprints/hurdles, Jumps, Lifting, Distance running, Conditioning
2. **Main goal:** Compete / get faster, Build muscle, Lose fat, Get fitter overall
3. **Events and PRs** (only if track or running was picked), which seed % of PR targets
4. **Training days:** which days, and roughly what kind of session each day
5. **Body + fuel goal:** weight, height, cut / maintain / gain
6. **How much fueling do you want?** Just protein / Full macros / Full meal plans

Screen 6 matters most: someone who only wants to log workouts shouldn't get pushed into meal planning. The result is a generated starter week the user can edit right away.

Build order and open questions

Build order, with each step shippable on its own:

1. Move storage to Supabase + SwiftData cache; migrate today's lift and meal records into Session → Block → Item.
2. Add the Sprint, Jumps, Hurdles and Plyo block types, with logging screens.
3. Day Load (Energy + Neural) and the day label in the Week view.
4. Fuel targets driven by Day Load; meal slots tied to sessions.
5. Performance table, PR badges and the share card.
6. HealthKit import, then Strava.
7. Website: week grid planner + meal plan builder on the same database.
8. Coach mode: assign programs to athletes (RLS policies).

Open questions:

- [ ] Should a Program/Block (mesocycle) layer above weeks be in v1, or is "copy last week" enough to start?
- [ ] Do a user's starting Neural thresholds come from a short onboarding question (sprinter, lifter, runner, hybrid) or from a default that adapts?
- [ ] Camera-based sprint timing: a later feature, or skipped in favor of manual entry?
- [ ] What's the minimum Strava/Health matching UI: auto-fill silently, or always confirm?